

1. 사용 모델 및 데이터셋

Dataset : SpeechCommands

음성 데이터셋으로, 1~2초 가량의 짧은 음성을 이용한 명령어 구분 Task Dataset이다. Clean한 음성데이터에 긴 길이의 환경 Noise 데이터의 일부 시간대 음성을 증강하여 데이터로 구성하였으며, 기존 데이터셋은 35개의 Class로 이루어졌으나, 신경망 구조 간편화 및 "yes", "no", "up", "down", "left", "right", "on", "off", "stop", "go"의 10가지 음성 Class를 선정하여 전처리하였다. 또한, 이전 프로젝트에서 수행하였듯, Waveform과 Mel-Spectrogram의 공격 자원의 형평성이 동등하지 않음을 인식하였기에, 이번엔 Mel-Spectrogram의 형식으로 변환하여 모델에 데이터를 입력하였다.

Model : AudioCResNet5

검증을 위해, 음성 ASR에서 주로 사용하는 CTC 베이스의 모델이 아닌, 단순한 ResNet 베이스의 Network를 구성하였다. 이 때, 파라미터 수를 동등하게 맞춰, (111862개) 기존 ResNet Base Model과의 차이점을 테스트해보려 하였다. 해당 Network를 10Epoch에 걸쳐 학습을 진행하였으며, 최종적으로 3703의 Validation Sample에 대해 Validation accuracy 0.9028의 결과로 신경망을 마련하였다.

```
loaded checkpoint: model/AudioCResNet5.pt
device: cuda
parameters: 111862
train_samples: 30769
val_samples: 3703
eval loss=0.2975 acc=0.9028
```

2. 결과와 결과 분석

1. α, β -crown의 동작

Case1) Unknown (Epsilon : 0.01)

BaB 단계에 진입한 후, 정해진 Timeout까지의 검증에서 반례를 찾아내지 못한다면, 이 데이터에 대해서는 적대적 예제가 확실히 검증되지 않았기에, Safe 혹은 Unsafe를 확인할 수 없는 단계로 판단할 수 있다. 이에 따라 Unknown이라는 별개의 Case로 관리된다.

```
BaB round 99
batch: 32
splitting decisions:
split level 0: [19, 229] [19, 229] [out-3, 990] [out-3, 990] [out-3, 522] [out-3, 431] [out, 1010] [out, 949] [out-3, 990] [out-3, 431]
best_l after optimization: -0.18777218461036682
beta sum per layer: [0.0, 31.500185012817383, 9.514059066772461, 46.6119270324707, 153.58670043945312, 0.0]
alpha/beta optimization time: 0.08908724784851074
pruning_in_iteration open status: True
ratio of positive domain = 36 / 64 = 0.5625
pruning-in-iteration extra time: 0.0021681785583496094
Time: get_branching_decision.update_bounds 0.0222 pickout 0.0127 branching_decision 0.0002 decision 0.0002 set_bounds 0.0035 prep
are 0.0037 transfer 0.0000 finalize 0.0010 func 0.1509 solve 0.1441 add 0.0011 bab_iteration 0.1592
Accumulated time: get_branching_decision.update_bounds 0.0222 bab_iteration 16.3929 pickout 1.0251 decision 0.0185 branching_decision
n 0.0137 set_bounds 0.3113 solve 14.9223 func 15.0599 prepare 0.4346 transfer 0.0013 finalize 0.1326 add 0.1057
max lb 0.909990747378752e-05 min lb -0.8171330438079834
Current (lb-rhs): -0.3452447295188904
Number of Verified Splits: 36 of 64
Current worst splitting domains lb-rhs (depth):
-0.34452 (6), -0.34318 (6), -0.33822 (6), -0.33689 (7), -0.32324 (6), -0.32074 (6), -0.31762 (6), -0.30953 (7), -0.29565 (6), -0.28869 (6), -0.2
8065 (7), -0.28473 (6), -0.27769 (7), -0.27483 (6), -0.26827 (7), -0.26823 (7), -0.26678 (6), -0.26501 (6), -0.26441 (7), -0.26411 (7),
6078 domains visited
Length of domains: 410
Cumulative time: 16.83988070487976

##### Summary #####
Final verified acc: 0.0% (total 1 examples)
Problem instances count: 1, total verified (safe/unsafe): 0, total falsified (unsafe/sat): 0, timeout: 1
mean time for ALL instances (total 1): 20.34293288124858, max time: 20.343136310577393
unknown (total 1), index: [0]
```

Case2) Safe (Epsilon : 0.001, 0.005)

Safe는 각 2단계, safe와 safe-incomplete로 나누는 것을 확인할 수 있었다. BaB 단계를 기준으로 판명이 되는데, safe-incomplete는 bab에 들어가기 전에 crown 또는 α -crown의 초기 bound propagation만으로 모든 출력 margin의 lower bound가 양수임이 확인된 경우이다. 즉, complete search 없이도 해당 epsilon ball 안에서 예측 class가 유지됨을 증명한 결과이다. 반면 safe는 초기 incomplete bound만으로는 충분하지 않아 BaB 탐색을 수행한 뒤, 남아 있던 불확실한 subdomain들을 모두 제거하거나 안전함을 증명한 경우이다. 실험에서는 작은 epsilon에서는 safe-incomplete가 비교적 빠르게 나타났고, epsilon이 커지면 초기 bound가 느슨해져 BaB 단계까지 진행되거나 제한 시간 안에 결론을 내지 못해 unknown이 발생했다. 따라서 safe-incomplete와, safe는 모두 안전성이 증명된 결과이지만, 후자가 더 검증하기 어려운 경우에 속한다고 볼 수 있다.

```
BaB round 21
batch: 10
splitting decisions:
split level 0: [25, 492] [25, 492] [out-3, 352] [25, 492] [25, 492] [25, 492] [25, 492] [out-3, 352] [out-3, 352] [out-3, 971]
all verified at 0th iter
best_l after optimization: 0.052806079387664795
beta sum per layer: [0.0, 0.0, 0.0, 0.3458147386442261, 0.0, 0.0]
alpha/beta optimization time: 0.018813236236572266
pruning_in_iteration open status: False
ratio of positive domain = 20 / 20 = 1.0
pruning-in-iteration extra time: 6.389617919921875e-05
Time: get_branching_decision.update_bounds 0.0227 pickout 0.0046 branching_decision 0.0001 decision 0.0002 set_bounds 0.0016 prep
are 0.0019 transfer 0.0000 finalize 0.0005 func 0.0171 solve 0.1333 add 0.0009 bab_iteration 0.1474
Accumulated time: get_branching_decision.update_bounds 0.0227 bab_iteration 2.6424 pickout 0.0963 decision 0.0030 branching_decision
0.0022 set_bounds 0.0349 solve 2.4930 func 2.5077 prepare 0.0427 transfer 0.0002 finalize 0.0140 add 0.0168
max lb 0.0066419243812561035 min lb 0.0014064908027648926
Current (lb-rhs): 1.0000000110860974e-07
586 domains visited
Length of domains: 0
Cumulative time: 2.9084315299987793

Result: safe in 4.6679 seconds
##### Summary #####
Final verified acc: 100.0% (total 1 examples)
Problem instances count: 1, total verified (safe/unsafe): 1, total falsified (unsafe/sat): 0, timeout: 0
mean time for ALL instances (total 1): 4.667891791922535, max time: 4.667938470840454
mean time for verified SAFE instances (total 1): 4.667938470840454, max time: 4.667938470840454
safe (total 1), index: [0]
```

```
##### Summary #####
Final verified acc: 100.0% (total 1 examples)
Problem instances count: 1, total verified (safe/unsafe): 1, total falsified (unsafe/sat): 0, timeout: 0
mean time for ALL instances (total 1): 0.28004509387323916, max time: 0.2800478935241699
mean time for verified SAFE instances (total 1): 0.2800478935241699, max time: 0.2800478935241699
safe-incomplete (total 1), index: [0]
```

Case3) Unsafe (Epsilon : 0.02)

Unsafe는, PGD Attack을 통해 적대적 예제가 발견되었음을 의미하며, 이는 해당 신경망이 Robust하지 않음을 나타낸다.

```
PGD attack margin (first 2 examples and 10 specs):
tensor([[[ 3.25552535, -5.49847422,  2.83714151, -0.19717026,  5.81523800,
           6.12384653,  4.89025354,  5.15089083,  5.46025658]])])
Total number of violation: 2
Attack finished in 0.0122 seconds.
PGD attack succeeded!

Checking and Saving Counterexample in check_and_save_cex
verified_status unsafe-pgd
verified_success True
Result: unsafe-pgd in 0.0289 seconds
```

3. α, β -crown과 Marabou의 차이점

4번 과제에서 더 깊이있는 신경망을 통해 검증하길 원했기에, 동등한 데이터셋과 모델에 대해 검증하지는 못했으나, 단일 데이터에 대해서, 조금 더 깊은 신경망을 사용하였지만, 검증속도가 유사함을 보일 수 있었으며, Marabou의 Non-Trivial Approach에서의 접근보다, α, β -crown의 BaB Approach가 (11round/s) 조금 더 빠른 검증이 가능하였으며, 실제로 심화적인 Safe Task를 풀이하는 데에 2초 가량의 시간만으로 11만개의 Parameter에 대해 강건함을 보일 수 있었다.

특히 신경망의 검증을 위해, 검증조건을 제약식으로 구체적인 설정을 진행해주어야 하는, Marabou에 비하여, α, β -crown을 이용한 검증은, 조금 더 간편한 편이었다. LiRPA 베이스라인을 기반으로 하기에, 심화적인 SMT 제약조건 없이 해당 신경망의 Robustness를 검증할 수 있음이 장점으로 느껴졌다.

4. α, β -crown의 강점 및 한계

이번에 수행한 실험에서의 α, β -crown의 강점이라면, 단연코 속도를 말할 수 있을 것이다.

Marabou에 비해, 굉장히 빠르고, 선제적인 Branch and Bound 알고리즘으로 가능성이 적은 파트를 잘라내기, 위 과정이 속도 상 이점을 가진다고 볼 수 있다. 또한 Marabou에 비해 GPU 병렬연산을 지원하기에, 데스크탑 환경에서는 굉장히 빠른 실행이 가능하였다.

실험에 따른 한계도 존재하였는데, 나의 데이터셋은 "yes", "no", "up", "down", "left", "right", "on", "off", "stop", "go"의 10가지 Class를 구분하는 Task이며, 그렇기에, Orthogonal한 그룹 간의 Robustness (yes - no, up-down)등의 차이점을 비교하는 Task를 설계하였을 때에는, α, β -crown으로는 해당 논리식을 충분히 담기엔 한계가 존재했다. 그에 반해 Marabou는, SMT Solver로서, 단순히 신경망 노드의 강건성뿐만 아니라, 결과물 간의 관계성을 모델링할 수 있어서, 해당 데이터 간 강건성을 보일 땐 Marabou가 더 유리함을 알 수 있다.

5. Dicussion

검증 데이터를 시각화하고, wav form으로 만들어서 청취하였더니, 예측 Class인 go가 아닌, down에 가까운 음성이 들렸다. 이에 대해 의구심이 들어 데이터를 분석하였더니, {GT : down} 으로 분류된 데이터에 대해, go로 분류한 것이었다. 처음엔, 이를 보고 α, β -crown를 사용하는 중 발생한 버그라고 판단하여 조치하려 하였으나, 이는 신경망이 GT에 맞지 않는 Prediction을 진행한 것이었고, α, β -crown은, 기존 오분류된 Prediction에 대한 공격을 진행, Class의 변화가 존재하지 않음을 판정 후 Safe로 결과를 낸 것이었다. 이에 따라, 이는 검증기의 문제가 아니며, 오히려 신경망의 예측이 잘못되었음에도 불구하고 Noise에 강건함을 보이는 증거로 작용할 수 있다.

